Method	Sparsity								Runtime (s)
	1	2	3	4	5	6	7	8	
Random	0.573	0.629	0.663	0.690	0.712	0.729	0.743	0.755	0.257
MMD	**0.832**	0.857	0.867	0.874	0.880	0.884	0.887	0.890	0.292
FS	**0.832**	0.856	0.867	0.874	0.880	0.884	0.887	0.890	0.303
LR	0.818	0.853	0.871	0.882	0.890	**0.897**	**0.902**	**0.906**	7.796
MI	0.831	0.851	0.863	0.869	0.876	0.880	0.884	0.886	32.241
OSP	0.831	**0.861**	**0.876**	**0.884**	**0.891**	0.896	0.899	0.902	53.028

Table 4: Comparison of sparse feature selection methods. Results are for the out-of-sample F1 score averaged for the best scoring layer for each combination of model and feature.

F-statistic We utilize a one-way ANOVA test to evaluate the relationship between the continuous data of each neuron independently and a discrete target. The F-statistic is calculated by partitioning the total variability into between-group and within-group components, represented by sum of squares between groups (SSB) and sum of squares within groups (SSW), respectively. The ratio of the mean square between groups (MSB) to the mean square within groups (MSW) is computed as the F-statistic. A higher F-statistic indicates a larger discrepancy between group means and is used as the score for each neuron.

Optimal Sparse Probing We train a cardinality-constrained support vector machine (SVM) with hinge-loss trained to provable optimality using the cutting planes technique as described in [17]. The selected coefficients are simply those selected by the classifier.

Adaptive Thresholding Starting from $k = d_{mlp}$ (or some large k on a filtered set of neurons from a different heuristic), we train a series of logistic regression probes with elasticnet regularization (that is combined l_1 and l_2 regularization). Given a schedule for reducing k, at each step t, we take the $k_t < k_{t-1}$ neurons with highest absolute coefficient magnitude and retrain a logistic regression probe on these neurons. This process enables us to sweep through many values of k while leveraging past computation to achieve better feature selection.

B.4 Experimental Procedure

Here we provide additional details on the experimental procedure regarding both the feature selection experiments and the sparsity with scale experiments within Section 4.

Perhaps most significantly, for all experiments we preceded probe training with a heuristic filtering step, where we only trained probes on the top 1024 neurons, as judged by mean difference, for each combination of layer, feature, and model. This was done for computational reasons, as computing the mutual information, training dense probes, and optimal sparse probing methods are very expensive to run on hundreds of features for many layers containing in excess of 10,000 neurons. For optimal sparse probing, we only used the top 50, and set a timeout of 60 seconds.

All classifiers were trained with balanced class weights to account for class imbalances in the dataset. Finally, we selected hyperparameters using a small subset of feature and model combinations, and then used the best performing hyperparameters for all other experiments, as doing seperate tuning for every trial would be too computationally expensive.

B.5 Feature Selection Results

For each combination of model, feature, and layer described above, we train a sparse probe for values of $k = 1, \ldots, 8$ using the features selected by the aforementioned subset selection methods. In Table 4, we report the out-of-sample F1 score of each method averaged across models and features, and for the maximum across layers since features tend to be only represented in specific layers.

While our results are averaging over many trials, several points stand out. The top methods are always within 1% of each other, with no method strictly dominating the others. Although OSP comes with optimality guarantees, we set a one minute timeout, which is frequently reached especially for larger values of k. Moreover, this provable optimality is with respect to the specific level of l_2 regularization which often needs to be turned fairly high to speed up convergence [17]. Nevertheless, it is impressive that such fast and simple heuristics as the mean difference are comparable to far more sophisticated approaches, with even random neurons containing substantial information of the target task. This suggests that perhaps the best design is a two-stage approach where very fast heuristics are used to find relevant neurons and layers, and optimal methods are used to improve and verify the solution.

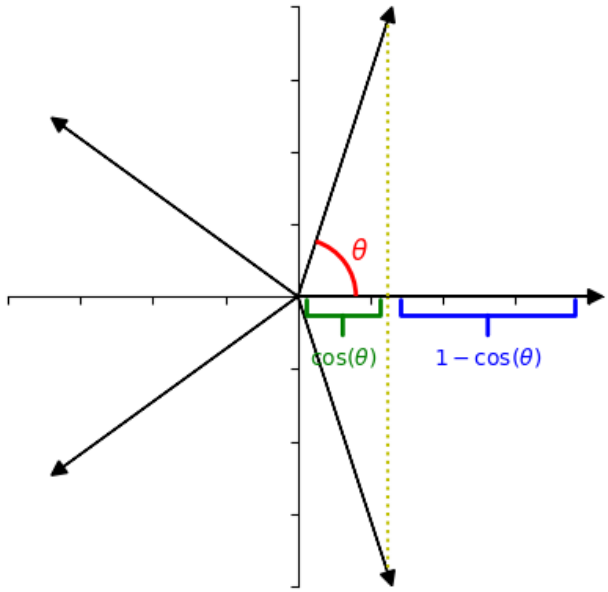

Figure 6: Intuition for representing 5 features in 2 dimensions in superposition.

C Superposition Construction

Consider a set of n binary and mutually exclusive features (i.e., one-hot vectors like n-grams), that we want to embed in two dimensions, and later losslessly recover. Specifically, given a one hot vector $x \in \mathbb{R}^n$, we seek a $W \in \mathbb{R}^{n \times 2}$ such that

$$x = \text{ReLU}(WW^T x + b)$$

where $\text{ReLU}(x) = \max(x, 0)$. By embedding each point to be equally spaced around a circle of radius α (i.e., $w_i = \alpha \cos(2\pi i/n), \alpha \sin(2\pi i/n)]$, we get that

$$(WW^T e_i)_j = \alpha^2 \cos\left(2\pi \frac{|i-j|}{n}\right)$$

where $\alpha = \|w_i\|_2$. To recover the initial one-hot representation, we require

$$\alpha^2 \left(1 - \cos\left(\frac{2\pi}{n}\right)\right) = 1 \quad \text{and} \quad b = -\alpha^2 \cos\left(\frac{2\pi}{n}\right)$$

Analyzing our proxy metric, $b\|w\|_2 = \frac{\cos(2\pi/n)}{(cos(2\pi/n)-1)}$ monotonically decreases for n with $n > 2$.

See [4] for a far more extensive set of experiments and results on related topics.

Note that this construction refers to residual stream superposition and *not* neuron superposition since we are assuming we have exactly as many neurons as features. However, we expect the same basic motif to clean up interference to hold for the case with fewer neurons than features. Moreover, it is plausible increased levels of residual stream superposition is associated with increased levels of neuron superposition. Hence we believe this construction, and therefore the presence of large weight norms and negative biases, to be suggestive evidence of neuron superposition.

Of course, it is possible factors other than superposition explain this observed weight pattern. Additionally, there might be multiple mechanisms for computing features in superposition (likely those that are not as binary or mutually exclusive as n-grams) that do not strongly affect the distributions of weights and biases. We would be very excited about future work that more carefully explained the source of these weight statistics, or that studied other concrete mechanisms or motifs enabling effective computation in superposition.

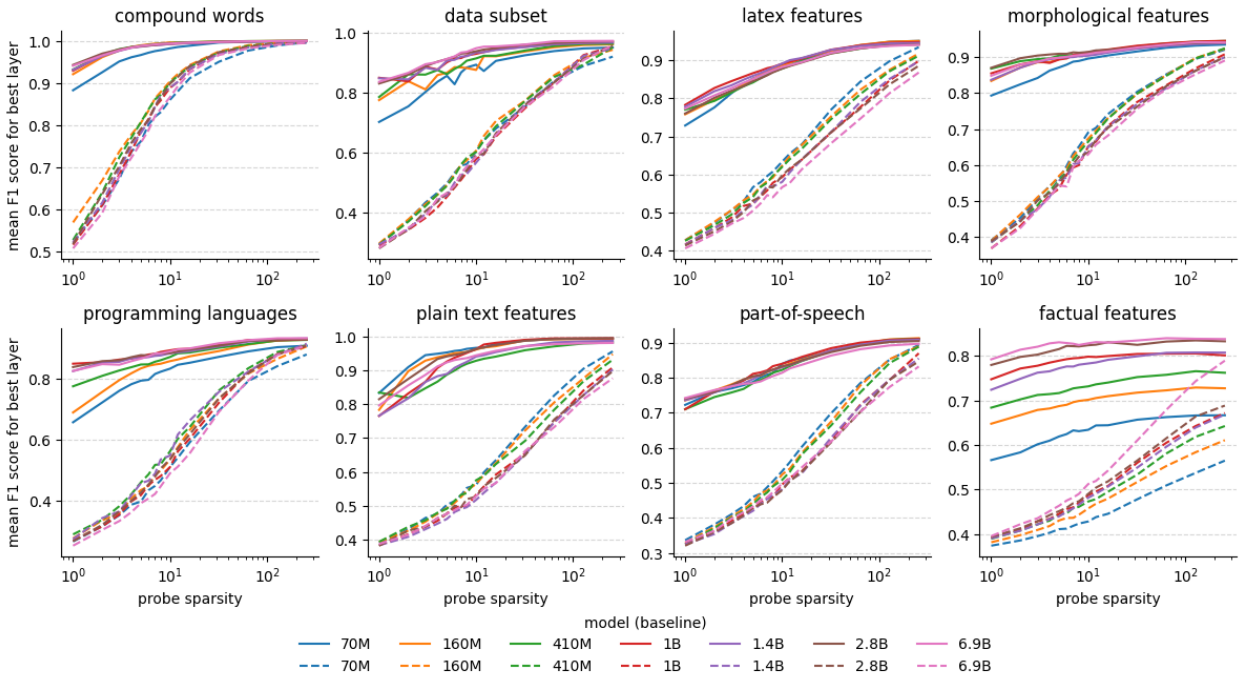

Figure 7: Difference between a k-sparse classifier using sparse features selection versus a random baseline. Specifically, the F1 score of training a classifier with k coefficients on the full $n \times d$ activation dataset when multiplied by a random $d \times k$ orthogonal matrix.

D Additional Results